

펌웨어 셋업 및 사용 안내 (Arduino Uno 보드 기준)

대회 요강상 **Arduino IDE**와 외부 라이브러리는 사용할 수 없습니다. 개발 환경은 Microchip Studio이고, 코드는 레지스터를 직접 read/write 합니다.

보드는 써도 됩니다. 요강 2-가-1이 "Arduino 보드"를 사용 가능 MCU로 명시하고 있습니다. 금지된 것은 IDE와 라이브러리입니다.

폴더 구조

```
firmware/  
  common/           모든 단계가 공유하는 모듈  
    pedd.c/h        PEDD 방전 시간 측정 코어  
    uart.c/h        UART 송신 (레지스터 직접 제어)  
    classify.c/h     보정 · 정규화 · 최근점 중심점 분류 [Phase 4]  
    store.c/h       EEPROM 중심점 저장 (EECR 직접 제어) [Phase 4]  
    i2c.c/h         TWI 마스터 [Phase 4]  
    ssd1306.c/h     OLED 텍스트 출력 [Phase 4]  
    font6x8.h       폰트 테이블 (tools/gen_font.py 생성) [Phase 4]  
    button.c/h      스위치 디바운스 [Phase 4]  
  01_blink/         튜체인 검증용 LED 점멸  
  02_phase1/        Phase 1 단일 채널 측정  
  03_phase2/        Phase 2 RGB 다파장 스캔  
  04_phase4/        Phase 4 최종 시스템 (버튼 + OLED + 온칩 분류)
```

각 단계는 별도 프로젝트입니다. `common/` 은 공유하므로 그대로 둡니다.

`classify` 는 하드웨어에 의존하지 않는 순수 계산 모듈이라 PC에서 단위 테스트합니다. `sh tools/run_host_tests.sh` 로 실행하며, `<avr/io.h>` 를 넣으면 이 테스트가 깨지니 주의하세요.

1. Microchip Studio 프로젝트 열기

프로젝트 파일은 이미 준비되어 있습니다. `firmware/PEDD.atsln` 을 더블클릭하면 `blink` · `phase1` · `phase2` · `phase4` 네 프로젝트가 한 번에 열리고, 아래 설정이 모두 들어가 있는 상태입니다. 빌드 검증도 마쳤습니다 (2026-09-03).

프로젝트	폴더	Flash	SRAM
blink	01_blink	176 B (0.5%)	0 B (0.0%)
phase1	02_phase1	1,430 B (4.4%)	89 B (4.3%)
phase2	03_phase2	1,360 B (4.2%)	121 B (5.9%)
phase4	04_phase4	6,634 B (20.2%)	90 B (4.4%)

빌드하려면 Solution Explorer에서 원하는 프로젝트를 우클릭 → **Set as StartUp Project** → **F7**. 결과물은 <폴더>\Debug\<이름>.hex 입니다.

프로젝트 파일을 새로 만들거나 고칠 일이 생기면 아래를 참고하세요. **손으로 만든 .cproj 는 프로젝트 폴더에 <이름>.componentinfo.xml 이 없으면 Value cannot be null. Parameter name: url 오류로 열리지 않습니다.** 이 파일은 디바이스 팩 경로를 담고 있으며, 같은 ATmega328P 프로젝트 것을 복사하면 됩니다. 또한 헤더 경로는 `../common` 이 아니라 `../../common` 이어야 합니다 (Makefile이 `Debug/` 폴더에서 실행되기 때문).

새로 만들어야 할 경우

1. **File > New > Project** → **GCC C Executable Project**
2. 디바이스 선택 창에서 **ATmega328P**
3. 자동 생성된 `main.c` 를 지우고 파일 추가 (Solution Explorer에서 프로젝트 우클릭 → **Add > Existing Item**) - `common/pedd.c`, `common/pedd.h`, `common/uart.c`, `common/uart.h` - 진행할 단계의 `main.c` 하나
4. 헤더 경로 추가 **Properties > Toolchain > AVR/GNU C Compiler > Directories** 에 `common` 폴더를 추가합니다. 이걸 빼먹으면 `pedd.h`를 찾을 수 없다는 오류가 납니다.

`01_blink` 는 `common/` 이 필요 없습니다. `main.c` 하나만 추가하세요.

F_CPU 설정 (필수)

`_delay_ms()` 와 UART 속도 계산이 이 값에 의존합니다. 잘못 두면 LED 점멸 주기가 어긋나고 시리얼 문자가 전부 깨집니다.

Properties > Toolchain > AVR/GNU C Compiler > Symbols → **Defined symbols (-D)** 에 추가:

```
F_CPU=16000000UL
```

최적화 옵션

Toolchain > AVR/GNU C Compiler > Optimization 에서 **-Os(기본값)**를 유지합니다. **-O0** 으로 두면 `_delay_ms()` 가 정상 동작하지 않습니다.

2. 보드에 업로드하기

Uno 보드에는 부트로더가 들어 있어서 **USB 케이블만으로** 업로드됩니다. 별도의 ISP 프로그래머가 필요 없습니다.

부트로더는 칩에 이미 들어 있는 업로드 수단일 뿐입니다. 요강이 금지한 Arduino IDE 라이브러리와는 무관하며, 코드는 여전히 Microchip Studio에서 레지스터 직접 제어로 작성합니다.

2-1. 준비물 2가지

CH340 드라이버 — 저가 Uno 호환보드는 대부분 CH340 USB 칩을 씁니다. Windows에서 자동으로 안 잡히면 "CH340 driver"로 검색해 설치하세요. 장치 관리자의 포트(COM & LPT)에 USB-SERIAL CH340 (COM3) 형태로 잡히면 성공입니다. 이 **COM 번호를 기억해두세요.**

avrdude — Microchip Studio에는 포함되어 있지 않은 별도 도구지만, 이 PC에는 이미 설치되어 있습니다 (winget AVRdudes.AVRDUDE 8.2, 2026-09-03). 새 터미널에서는 avrdude 로 바로 호출되고, 전체 경로는 아래와 같습니다.

```
C:\Users\hwali\AppData\Local\Microsoft\WinGet\Packages\AVRDudes.AVRDUDE_Microsoft.Winget.Source_8wekyb3d8bbwe\avrdude.exe
```

2-2. Microchip Studio에 업로드 버튼 만들기

Tools > External Tools... → Add

항목	값
Title	Uno Upload
Command	avrdude.exe 의 전체 경로
Arguments	-c arduino -P COM3 -b 115200 -p m328p -U flash:w:"\$(ProjectDir)Debug\\$(TargetName).hex":i
Initial directory	\$(ProjectDir)

COM3 부분을 2-1에서 확인한 실제 번호로 바꾸세요. 이제 빌드(F7) 후 Tools > Uno Upload 로 업로드합니다.

업로드 직전 보드가 자동 리셋되면서 몇 초간 시리얼 포트를 점유합니다. 터미널 프로그램을 열어둔 채로 업로드하면 실패합니다. 터미널을 닫고 업로드한 뒤 다시 여세요.

3. 단계별 진행

01_blink — 톨체인 검증

배선이 필요 없습니다. 보드의 "L" LED(PB5)가 1초 주기로 깜빡이면 통과입니다. **이게 될 때까지 다음으로 넘어가지 마세요.** 여기서 막히는 원인은 대부분 COM 포트 번호나 F_CPU 설정입니다.

02_phase1 — 단일 채널 측정

배선은 02_phase1/main.c 상단 주석을 따릅니다.

측정 순서 (계획서 4.1.3절)

- 1. 빈 챔버(암실) — 암전류 기준. Overflow Counter 동작 확인
- 2. 증류수
- 3. 착색수
- 4. 희석 우유(탁도)
- 5. 광 경로 완전 차단 → TIMEOUT 반환 확인

각 조건마다 10회 이상 반복하고, 시료별로 로그 파일을 나눠 저장합니다.

03_phase2 — RGB 다파장 스캔

발광 LED 3개를 PB0/PB1/PB2(D8/D9/D10)에 연결합니다. 한 사이클에 DARK → R → G → B 네 행이 출력되며, 같은 seq 번호로 묶입니다.

분석 스크립트가 이 seq를 이용해 광학 지문 벡터를 재구성하고, **R → RG → RGB 순으로 채널을 늘렸을 때 시료군 분리도가 개선되는지**를 계산합니다. 계획서 4.2.4절의 성공 기준이 이것입니다.

04_phase4 — 최종 시스템

PC 없이 동작합니다. 배선은 phase2에 스위치(D2)와 OLED(A4/A5)를 더한 것입니다.

조작	동작
짧게 누름	측정 후 판정
2초 길게 누름	학습 모드 — 3종을 순서대로 측정해 EEPROM에 저장

처음 켜면 NOT TRAINED 가 뜹니다. **길게 눌러 학습을 먼저 해야 판정이 됩니다.** 학습 도중 길게 누르면 취소되고 기존 학습 데이터가 보존됩니다.

측정값은 OLED와 UART로 동시에 나갑니다. **OLED가 없거나 배선이 틀려도 UART 로깅은 계속됩니다.** 실험 중 디스플레이 문제로 데이터 수집이 멈추면 안 되기 때문입니다.

부품 도착 후 연결 순서는 docs/superpowers/plans/2026-09-03-phase4-bringup.md 체크리스트를 따르세요. 한 번에 다 연결하면 어디가 문제인지 알 수 없습니다.

3-1. 배선표 (회로도 → 우노 보드)

회로도(`figures/fig3_schematic.svg`)는 ATmega328P 단품 기준으로 그려져 있습니다. 회로도의 부품 대부분은 우노 보드에 이미 실장되어 있으므로 배선할 필요가 없습니다.

보드에 이미 있는 것 — 손대지 않음

회로도 부품	역할
U1	ATmega328P 본체
Y1, C4, C5	16MHz 클럭
C1, C2, C3	전원 디커플링
R4	RESET 풀업
J1	UART — 보드의 USB 단자가 대신함

브레드보드에 실제로 배선하는 것

회로도	우노 핀	연결
R1 + D1 (적색)	8	핀 8 → R1 → D1 애노드, D1 캐소드 → GND
R2 + D2 (녹색)	9	핀 9 → R2 → D2 애노드, D2 캐소드 → GND
R3 + D3 (청색)	10	핀 10 → R3 → D3 애노드, D3 캐소드 → GND
D4 (검출, 적색)	7	D4 캐소드 → 핀 7, D4 애노드 → GND
SW1	2	핀 2 → 스위치 → GND (외부 저항 없음)
J2 OLED	A4 / A5	SDA→A4, SCL→A5, VCC→5V, GND→GND (Phase 4)

⚠ 회로도의 D1~D4는 LED 부품기호이고, 위 표의 숫자는 보드의 디지털 핀 번호입니다. 서로 다른 것입니다. 예를 들어 D2는 녹색 LED를 가리키고, 핀 2에는 스위치가 붙습니다. 혼동해서 녹색 LED를 핀 2에 꽂지 마세요.

⚠ D4만 극성이 반대입니다. D1~D3은 핀 → 애노드(긴 다리)로 연결하지만, D4는 핀 7 → 캐소드(짧은 다리, 몸통이 꺾인 쪽)로 연결합니다. 반대로 꽂으면 순방향이 되어 전하가 축적되지 않고 측정이 아예 성립하지 않습니다.

GND는 보드의 어느 GND 핀에 연결해도 됩니다 (POWER 헤더에 2개, 디지털 13번 옆에 1개).

4. 시리얼 로그 받기

터미널 프로그램(PuTTY, Tera Term)을 38400 bps, 8N1로 엽니다.

```
seq,ch,status,ticks,us,ovf
0,DARK,OK,412880,206440,6
0,R,OK,1204,602,0
0,G,OK,980,490,0
0,B,OK,1510,755,0
```

터미널의 로그 저장 기능으로 파일에 받은 뒤 분석합니다.

```
python tools/analyze.py data/distilled.csv data/dye.csv data/milk.csv
```

5. 자주 막히는 지점

증상	원인	해결
avrdude: can't open device "COM3"	COM 번호 오류, 터미널이 포트 점유 중	장치 관리자에서 번호 확인, 터미널 닫기
programmer is not responding	보드 리셋 타이밍	업로드 명령 실행 직후 보드의 RESET 버튼을 한번 누름
pedd.h를 찾을 수 없음	헤더 경로 미설정	Toolchain > Directories에 common 추가
시리얼 문자가 깨짐	F_CPU 불일치, 통신 속도 오설정	F_CPU=16000000UL 확인, 터미널 38400 확인
방전 시간이 항상 0	검출 LED 극성 반대	캐소드가 PD7인지 확인
Red 채널만 TIMEOUT	검출 LED가 청색·녹색	적색 검출 LED로 교체
모든 측정이 TIMEOUT	검출 LED 극성 반대, 광량 부족	극성 확인 → 발광·검출 거리 축소 → 저항값 하향
측정값이 매번 크게 변동	외부광 유입, 광학 경로 흔들림	차광 챔버 밀폐, LED·큐벳 고정 강화